to evaluate if one multi-byte string is ordered above or below another: `strncmp()` and `strncasecmp()` are not aware of encoding. Hopefully by now this warning will be redundant, but here it is anyway: remember that "case-insensitive" `strncasecmp()` only applies to ASCII range characters.

```
var_dump(strncasecmp("iBanEz JeM", "IbAnEZ rG", 6));
var_dump(strncasecmp("Zebedee", "\u{141}ukasz", 2));
```

```
Output:
int(0)
int(-75)
```

In the second line, the name `'Łukasz'` should be ordered before `'Zebedee'` (assuming we want the ordering to respect the Polish alphabet: more on this in just a moment), but the return value `-75` places `'Zebedee'` before `'Łukasz'`. Note that `strncasecmp()` return values deviate slightly from `strncmp()` (which only returns `-1`, `0`, or `1`); this doesn't affect its use as a sorting function, but be aware you'll need to account for it if used in a logical expression. (Interestingly, the official PHP documentation states that `strncasecmp()` should only return `-1`, `0`, or `1` since PHP 8.2, but this is not the case.)

substr_compare()

The `substr_compare()` function is very similar to `strncmp()`but allows you to specify an offset and an optional length and can be set to case-insensitive comparison (again, this only works properly for ASCII strings). It will compare a sub-string of bytes from the first argument (as per the offset and length) to the second (up to the specified length), returning a negative or positive number if the comparison is not equal, and zero if it is.

```
$mijRG = 'Ibanez RG 550 Genesis'; // seriously good VFM this one ;)

// signature is substr_compare($string1, $string2, $offset, $length = null,
$caseInsensitive = false);
// search 'Ibanez' for 'Ibanez'
var_dump(substr_compare($mijRG, 'Ibanez', 0, 6));
// search 'Ibanez' for 'RG 550'
var_dump(substr_compare($mijRG, 'RG 550', 0, 6));
// search 'RG 550' for 'RG 550'
var_dump(substr_compare($mijRG, 'RG 550', 7, 6));
```